

邓泽霖

AI 工程师 · Agentic Workflow 方向 · 4 年大厂经验 · 连续 2 年 A 绩效
188-4617-4594 · 2597419838@qq.com · 个人主页 · github.com/zachary-lz-glm



个人优势

4 年大厂经验,连续 2 年 A 绩效。专注 LLM 工程化,基于 Claude Code / LangGraph 独立设计并落地多套 Multi-Agent 系统,核心方向:Agent 编排架构 / Agent Safety / RAG 工程。

- **Spec-Driven Development Platform** — 用 **Evidence-based Grounding** (file:line citation) + **Distilled Knowledge Base** 解决 LLM 编码 Hallucination;需求理解 **5h → 2h** / 需求,20+ 人团队日常使用。Ablation study 验证 grounding 层为必需(rubric 评分 **82 → 54**)。
- **Manager Agent** — **39 节点** LangGraph Supervisor 状态机编排 12 个能力 Agent,Plan-and-Execute + Critic 闭环 + HITL Checkpoint + Probe + Health 双层探活 + Self-Evolution Layer。
- **code assistant Agent** — ReAct Tool Use 循环(**18 工具**)+ Custom CheckpointSaver + Agent Safety(三层写盘守卫 + Sandbox Isolation)+ Prompt A/B(Shadow Patch)。
- **Server-Driven UI** 中台 + **400+** 权益组件库 — 4 年大厂业务落地:覆盖 10+ 国家 / 20+ 活动类型 / 8 业务项目,新权益接入 **2 天 → 2 小时**。

专业技能

Agent 架构:LangGraph StateGraph / Checkpointer · Supervisor / Orchestrator 模式 · Plan-and-Execute(Planner / Critic / Synth)· ReAct Tool Use · Sub-agent · MCP 协议 · Function Calling

Agent Safety & Self-Evolution:Sandbox Isolation · Read-before-Write / Diff 工程 · HITL Checkpoint · Probe + Health 双层探活 · Self-Evolution(online learning loop)

RAG / Context Engineering:Hybrid Retrieval(BM25 + Dense Vector + Reranking)· 代码 RAG / 仓库语义检索 · Distilled Knowledge Base · Context Engineering · Citation Grounding

Eval & Prompt 工程:rubric-based Eval · Ablation Study · Golden Set · Prompt A/B(Shadow Patch)· LLM-as-a-Judge

前端工程:React + TypeScript · Node.js Serverless BFF · Server-Driven UI / Schema-as-Code · Monorepo(Lerna + Nx)· WebSocket 流式协议 · 微前端(qiankun)· CI/CD

工作经历

滴滴出行 · 国际化事业部 | AI 工程师 · Agentic Workflow 方向 | 2022.06 - 至今

2024 年起专注 LLM 工程化,基于 Claude Code / LangGraph 独立设计并落地 **3 套 Multi-Agent 系统** (Spec-Driven / Manager Agent / code assistant Agent,详见项目段),覆盖 Spec → Plan → Code SDLC 自动化 / Multi-Agent 编排 / Coding Agent Safety 三个方向。此前 2 年作为前端 owner 主导国际化营销中台,Server-Driven UI 架构 + 400+ 权益组件库服务 10+ 国家 / 8 业务项目,标准活动上线 **2 周 → 3 天**。连续 2 年 A 绩效。

项目经历

I Spec-Driven Development Platform — Spec-Driven Development / Agentic Coding workflow

独立设计并主导落地 · Claude Code Skill · Context Engineering · Citation Grounding · Sub-agent

业务场景:B 端 PRD 到代码缺少可追溯中间产物 Spec,AI 直接分析 PRD 缺历史代码上下文与隐性业务逻辑,产出计划不可执行、返工频发。本平台基于 Claude Code + MCP 用 Agentic RAG + Citation Grounding 解决企业级 AI 编码痛点。

核心技术:

- **Distilled Knowledge Base + Hybrid Retrieval** — 历史代码 / PRD / 跨层契约蒸馏为 6 类知识(代码结构 / 编码规则 / 契约 / PRD 路由 / 业务领域 / 实体索引),倒排索引 + 实体图按需检索(Context Engineering 范式)
- **Plan-and-Execute** 闭环 — 一次蒸馏产出三份产物(开发计划精确到行号 + QA Matrix + Rollback Plan)+ Final Quality Gate(5 维准出报告:完整性 / 一致性 / 可追溯 / 可行性 / 风险),区别于单点 PRD → Plan 工具
- **Evidence-based Grounding** — 每条事实附带 file:line citation,源码 Read 后才写入,禁止推断,降低 LLM 编码 Hallucination
- **Spec Review Gate** — Spec 必须用户 approved 才能进入 Plan(HITL gate before plan execution),避免错误计划被开发执行

量化产出:需求理解阶段耗时 **5h → 2h** / 需求 (n=5 PRDs);文件级覆盖率 80%+;同一 PRD 蒸馏 determinism **40% → 85%+** (n=10) ;20+ 人团队日常使用,后端基于 Spec 产物扩展。

Ablation Study:去掉 grounding 层,rubric 评分(完整性 / 一致性 / 可追溯 / 可行性 / 风险,满分 100)**82 → 54**。

I Manager Agent — Multi-Agent Supervisor Orchestrator

主导设计实现 · LangGraph · Multi-Agent · HITL · Self-Evolution · Plan-and-Execute

业务场景:企业级 AI 助手需统一调度多个专业 Agent(知识库 / NL2SQL / 代码 / 爬虫 / GUI / 办公 / 多模态),“一个 Prompt 包打天下”无法支撑。本系统基于 LangGraph 设计 Supervisor 编排模式,调度 12 个能力 Agent (8 外部 + 4 内部)。

核心技术:

- **Multi-Agent Supervisor** 编排 — 基于 LangGraph 设计 39 节点 状态机,编排层与专家层分离(总管只做意图路由 / 任务规划 / 结果汇总,不写 SQL / 不建向量 / 不改代码);支持 14 路意图分发 与子句拆解(多意图 → 多步 plan)
- **Plan-and-Execute + Critic** 闭环 — Planner 拆任务 → Executor 并行调度 → Critic 事实性审查 → Synth 综合产出;Critic 与 Evaluator 形成“重试否决”仲裁,避免无谓重试;recursionLimit 48 内长跑稳定
- **Probe + Health** 双层探活 + HITL 人工确认 — 规划前先探针下游能力(RAG 文档 / DB schema),失败则降级跳过 down 的 Agent;高风险写操作 HITL 人工确认,file + Redis 双后端 checkpoint 支持跨进程续跑
- **Self-Evolution Layer** — 在线学习闭环 — ① 失败归因 12 分类 → ② LLM 假设生成(每 30min 产出 ≤3 条) → ③ 假设金丝雀 A/B 验证命中后晋级 Prompt 补丁 → ④ 经验回放(向量 + Jaccard 混合召回)注入 router few-shot,整套机制无需人工标注

量化产出:39 节点 / 42 条边 (含 14 条条件边) · 12 Agent (8 外部 + 4 内部) · 8 类 WS 消息 · 213 个 TS 文件(约 4.6 万行) · 7 个 eval golden + 34 个 smoke 脚本。

I code_assistant Agent — Agentic Coding / Coding Agent Safety

主导设计实现 · LangGraph · ReAct · Tool Use · 代码 RAG · Agent Safety

业务场景:LLM 改仓库的高风险场景下“模型 + 写权限 = 事故”。本 Agent 把 LLM 改仓库约束在可审计的工程边界内,完整覆盖 Read-before-Write → 语义检索 → 静态分析 → Diff 预览 → 受控写盘 工程闭环。

核心技术:

- **ReAct Tool Use** 工具循环 — 基于 LangGraph 编排 18 个 仓库工具(读 / 检索 / AST 分析 / Patch / 校验 / 执行 / Git)的 agent↔tools 循环;Custom CheckpointSaver 实现 BaseCheckpointSaver 接口,支持跨进程会话状态恢复 (LangGraph 早期版本仅内置 MemorySaver)
- 代码 **RAG + Unified Diff** 工程 — 仓库语义检索(向量召回 + 文件名粗筛 + 增量索引),区别于端到端 chat 把全量 repo 塞 context;统一 Diff 预览再写入,read-before-write,杜绝 LLM 错误 diff 静默覆盖
- **Agent Safety** — 三层写盘守卫 + **Sandbox Isolation** — 路径黑名单 + 受保护目录 + sha256 乐观锁,任一层拦截即拒绝;subprocess 白名单 env + docker 网络隔离双沙箱,优先强隔离不可用时自动降级
- **Auto-Validation Hook + Prompt A/B(Shadow Patch)** — autoValidateAfterWrite 自动跑 typecheck / lint / test,失败针对性修复,信号回流 Prompt 进化库;shadow patch 多次命中晋级正式库,hash 分桶 A/B 对照

量化产出:18 个工具 覆盖 read / search / vector / AST / patch / validate / exec / git 全链路 · 默认服务 800 文件仓库 / 512MB 内存 / 90s 超时 · 跨进程会话恢复(回归测试 0 丢失)。

I Server-Driven UI 中台 + 400+ 权益组件库

前端 + BFF 架构设计 · Server-Driven UI · Plugin Architecture · Schema-as-Code · React · TypeScript

业务场景:国际化营销中台覆盖 10+ 国家、20+ 活动类型、400+ 种营销权益(优惠券 / 折扣 / 积分 / 立减)全生命周期。传统模式每新增权益前后端各开发一套,周期长易错。

核心技术:

- **Server-Driven UI + Plugin Architecture** — Schema 化页面指令驱动运行时渲染,权益作为插件通过 SDK 注入,新增权益无需改 SDK / 无需发版,接入从 2 人天降到 2 小时
- **Schema-as-Code + DSL** 引擎 — DSL 描述权益规则,引擎做两阶段求值(编译期 schema 校验 + 运行期求值),解决权益规则跨国家 / 跨活动动态组合问题
- **Plugin Registry + Hot Swap** — 35+ 渲染组件通过 Registry 注册,发布订阅驱动运行时渲染,组件 hot-swap 不停机

量化产出:新活动 Server-Driven 接入,前端零配置;标准活动上线 2 周 → 3 天 (联调 + QA 时间);权益库 400+ SKU 服务 8 个业务项目(底层组件 30-40 个),新权益接入 2 天 → 2 小时,运行 18 个月+ 仅 2 起低危 bug。

教育背景

黑龙江大学 | 计算机科学与技术 | 本科 | 2018.09 - 2022.06